browser, and create damage to any extent possible.

SQL injection: As we saw earlier, Web portals use database servers in the back-end, whereby the Web page connects to the database, queries for data, and presents the fetched data in a Web format to the browser. SQL injection attacks can occur if the input on the client side is not filtered appropriately before it is sent to the database in a query form. This can result in the possibility of manipulating SQL statements, in order to perform invalid operations on the database. A common example for this attack would be of an SQL server, which is accessed by a Web application, wherein the SQL statements are not filtered by middleware or validation code components. This can lead to the hacker being able to craft and execute his own SQL statements on the back-end database server, which could be simple SELECT statements to fetch and steal data, or could be as serious as dropping an entire data table. In other cases, the data can be corrupted by populating record sets with malicious and fake content. Despite the increasing awareness about cyber security, SQL injection attacks are still possible on many websites.

While it is impossible to cover all the possible attacks in this article, let us take a look at a couple of lesser-known attacks, which are increasingly being used to exploit websites.

Slow HTTP attack: While this one is similar to the denial of service attack, the technique is a bit different. It exploits the fact that each HTTP request must be listened to by the Web server. Every Web request starts with a field named *content-length*, which tells the server how many bytes to expect, and terminates with a carriage-return and line-feed (CRLF) character combination. The HTTP request is initiated by the hacker with a large value for *content-length*, and instead of sending the CRLF to conclude the request, it is simply delayed by sending very small amounts of data to the Web server. This makes the Web server wait for more data that is yet to come, to complete the request. This consumes Web server resources. If the request is delayed to a point that is just less than the session time-out setting on the server, multiple such slow requests can completely consume resources and create a denial of service attack. This can be achieved merely by creating slow and delayed requests from one single browser, which makes it dangerous from the security perspective.

Cryptographic exploitation: Secure websites use SSL certificate-based technology to encrypt data flowing over the network. This leads to an illusion that everything is safe, which is unfortunately not the case. Many shopping-cart applications forget to further encrypt the cookie contents, and leave those in plain text. Though the data over the wire is protected by SSL, running a client-side script to intercept the cookie and read its contents can potentially result in data or session theft. As for SSL, modern hackers use tools to detect and break into weaker cipher algorithms, thus rendering SSL protection useless, though this is not very common.

Protecting FOSS systems

Apache running on Fedora and Ubuntu has gained immense popularity among serious FOSS Web infrastructures and solutions. The very first step is to harden the Apache Web service itself; there are numerous guides and examples on the Internet regarding that—for each Linux distro, along with examples. Disabling ports other than the Web service port, and stopping and disabling unnecessary services is highly recommended. Deploying a well-configured firewall or intrusion-detection device is essential. As mentioned earlier, a simple firewall is not sufficient; hence, a content-filtering firewall equipped to detect Web layer attacks is required. Securing Web portals is not limited to the Web server, but also extends to components such as database servers, Web services, etc. From the network security stand-point, allowing IP connections to the database only from front-end Web servers is a good idea. Running root-kit detectors, anti-virus tools and log analysers must be a routine job, to prevent hacking attempts. For advanced security between the middleware and Web server, a stronger authentication mechanism should be in place too. Cookies should be encrypted and SSL deployed, with stronger cipher algorithms.

From the coding perspective, as we learnt earlier, it is essential to use secure programming techniques, and also to follow the best security practices, such as code reviews and penetration testing. Additional processes such as input code validation, server and database-side validation, is recommended too. Web exploitation is a common way of attacking websites. Due to its easy availability and programmability, FOSS infrastructure is also susceptible to such attacks—and hence, network administrators must understand techniques to protect their infrastructure from information loss or theft. END

By: Prashant Phatak

The author has over 18 years of experience in the field of IT hardware, networking, Web technologies and IT security. He is an MCSE, is MCDBA certified and is also an F5 load balancer expert. In the IT security world he is an ethical hacker and Net-forensic specialist. Prashant runs his own firm named Valency Networks in India (http://www.valencynetworks.com/) providing consultancy in IT security design, security audits, infrastructure technology and business process management. He can be reached at *prashant@valencynetworks.com*.